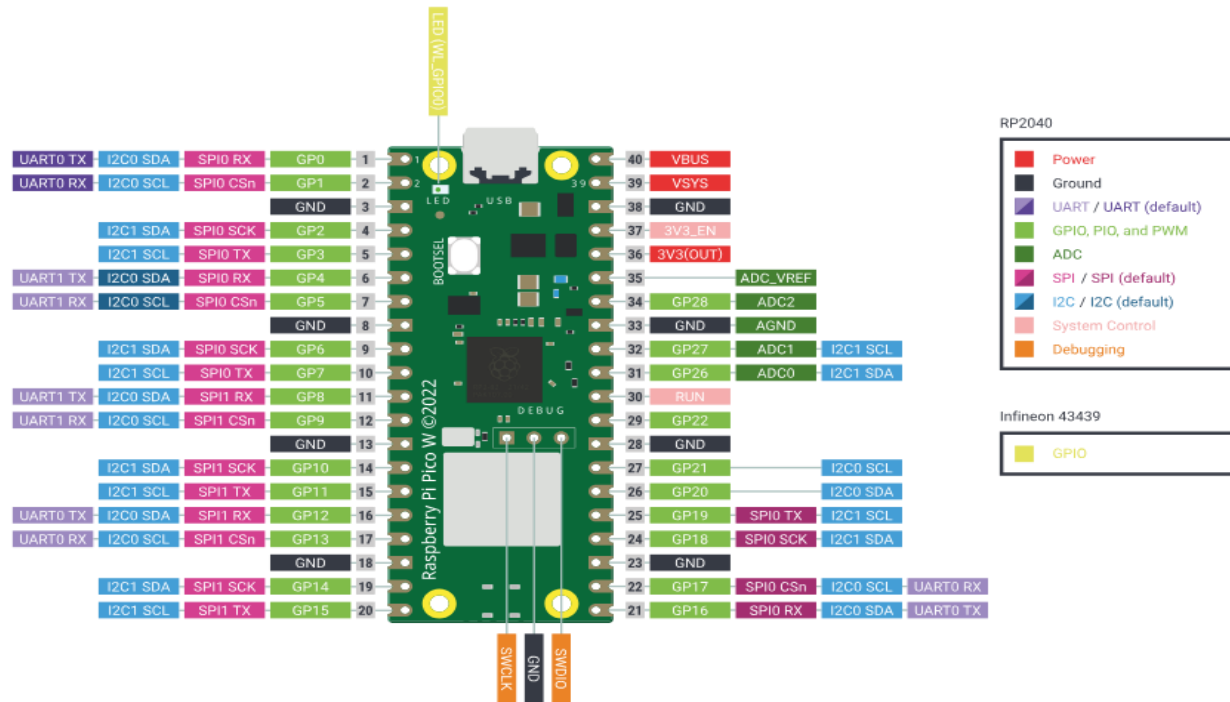


Sink current and Source current

RASPBERRY PI-PICO-W SERIES



GUIDED BY:

DURAI P S

MENTOR:

DANIEL RAJ A

TEAM:

KARTHICK A

LUCKSHANA V R

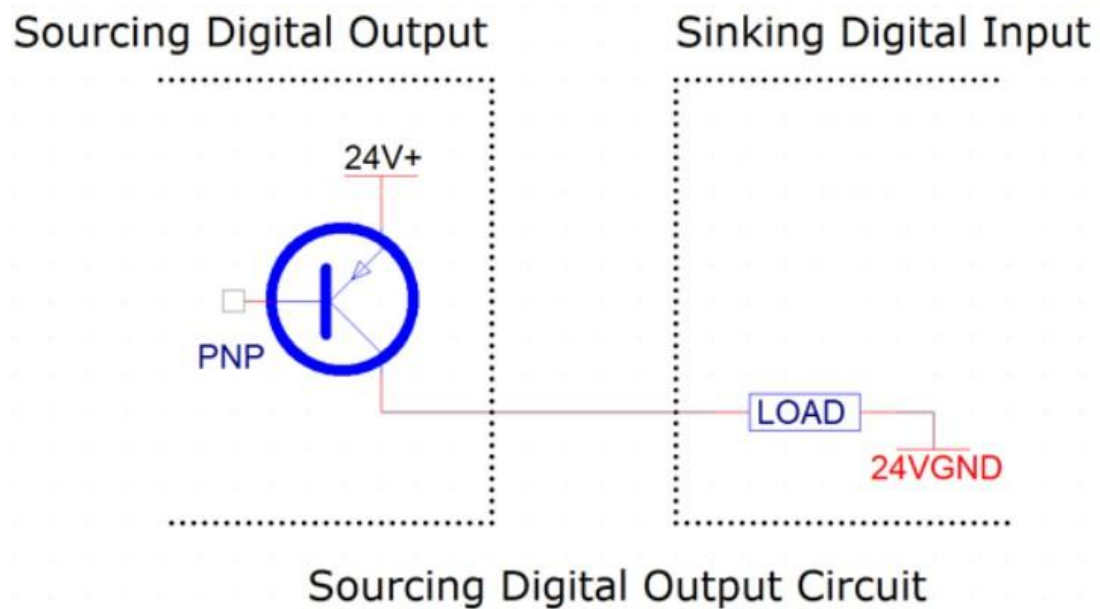
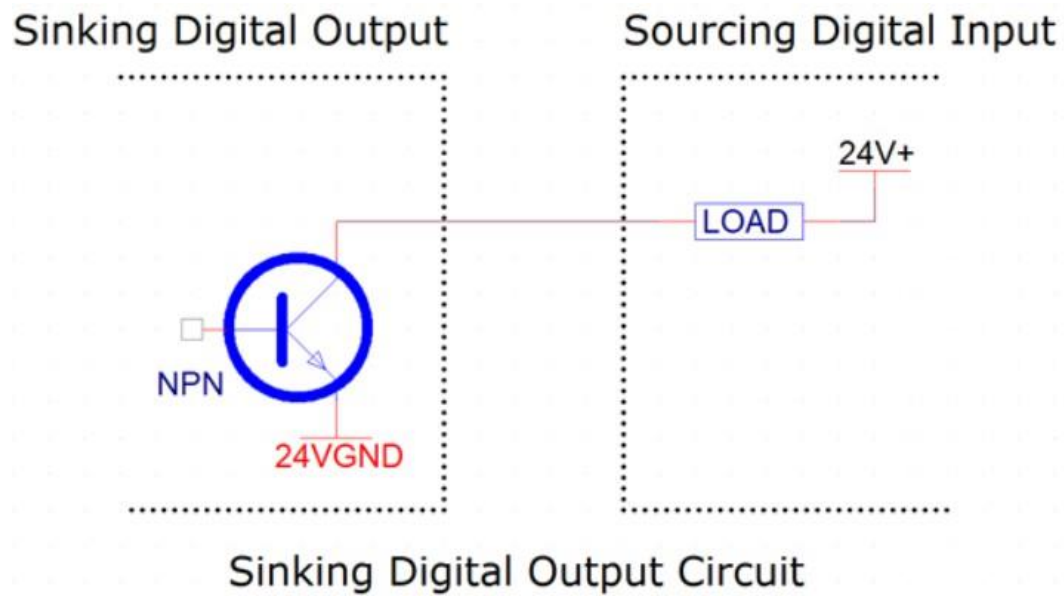
SHREE SUBI G

JANASRUTHIKA P R

TABLE OF CONTENT

S.no	Content	Page number
1.	<u>Definition of sink current and source current</u>	<u>4</u>
2.	<u>Sink current</u>	<u>5</u>
3.	<u>Source current</u>	<u>9</u>
4..	<u>Appendix</u>	<u>11</u>

SINK CURRENT AND SOURCE CURRENT:



1. Definition of sink current and source current:

Features	Source Current	Sink Current
Definition	Current flows out of the microcontroller pin	Current flows into the microcontroller pin
Pin Output State	GPIO is set to HIGH	GPIO is set to LOW
Current Flow Direction	From MCU pin → device → GND	From Vcc → device → MCU pin
Max Current	Often 10–20 mA	Often 20–40 mA
Acts Like	Current supplier (like Vcc)	Current receiver (like GND)
Limitation	Lower current capacity than sink	Can usually handle more current than source
Example Circuit	LED anode to pin, cathode to GND	LED anode to Vcc, cathode to pin

1.1. REGISTERS USED IN THE CODE:

SIO	Software control of GPIO, from the single-cycle IO (SIO) block. The SIO function (F5) must be selected for the processors to <i>drive</i> a GPIO, but the input is always connected, so software can check the state of GPIOs at any time.
-----	--

0x020 | GPIO_OE | GPIO output enable

0x014	GPIO_OUT_SET	GPIO output value set
0x018	GPIO_OUT_CLR	GPIO output value clear

2. FOR SINK CURRENT:

2.1. SET UP PICO_SDK ENVIRONMENT:

a. Create a working directory

```
$ mkdir pico
$ cd pico
$ mkdir blink_sink
$ git clone -b master https://github.com/raspberrypi/pico-sdk.git
$ export PICO_SDK_PATH=~/pico/pico-sdk
$ cd blink_sink
```

- Circuit Connection:
LED Anode - 3.3V (with 330k ohm resistance)
LED Cathode - GPIO14

b. Create CMakeLists.txt inside `~/pico/blink_sink` directory:

```
$ nano CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.13)
include(pico_sdk_import.cmake)

project(blink_sink C CXX ASM)
pico_sdk_init()

add_executable(blink_sink blink_sink.c)
target_link_libraries(blink_sink pico_stdlib)
pico_add_extra_outputs(blink_sink)
```

c. Create blink_sink.c(For Source current):

```
$ nano CMakeLists.txt
```

```
#include "pico/stdlib.h"
#include <stdint.h>

// Bitmask for GPIO14
#define LED_PIN 14
#define LED_MASK (1u << LED_PIN)

// Direct register access to SIO
#define SIO_GPIO_OE (*(volatile uint32_t*)(0xd0000020))
#define SIO_GPIO_OUT_CLR (*(volatile uint32_t*)(0xd0000018))
#define SIO_GPIO_OUT_SET (*(volatile uint32_t*)(0xd0000014))

int main() {
    // Initialize GPIO14
    gpio_init(LED_PIN);
    gpio_set_dir(LED_PIN, GPIO_OUT);

    // Enable output via SIO
    SIO_GPIO_OE |= LED_MASK;

    while (1) {
        // Sink current: pull GPIO14 LOW to turn LED ON
        SIO_GPIO_OUT_CLR = LED_MASK;
        sleep_ms(1000);

        // Set GPIO14 HIGH to turn LED OFF
        SIO_GPIO_OUT_SET = LED_MASK;
        sleep_ms(1000);
    }
}
```

Sink current and Source current

- d. Add `pico_sdk_import.cmake`, This file lets CMake find the SDK(Give your correct path):

```
cp ~/pico/pico-sdk/external/pico_sdk_import.cmake ~/pico/blink_sink/
```

- e. Build the Project

```
$ cd ~/pico/blink_sink  
$ mkdir build  
$ cd build  
$ cmake ..  
$ make
```

- f. Flash into Pico W

- Hold the **BOOTSEL** button and plug in the Pico W.
- It should mount as a USB mass storage.
- Copy the `.uf2` file:

```
$ cp blink_sink.uf2 /media/$USER/RPI-RP2/
```

3. FOR SOURCE CURRENT:

- Circuit Connection:
LED Anode - GPIO14 (with 330k ohm resistance)
LED Cathode – Ground

g. Create CMakeLists.txt inside `~/pico/blink_sink` directory

h.

```
$ nano CMakeLists.txt
```

```
cmake_minimum_required(VERSION 3.13)

include(pico_sdk_import.cmake)
project(source_current C CXX ASM)

pico_sdk_init()

add_executable(source_current source_current.c)
target_link_libraries(source_current pico_stdlib)
pico_add_extra_outputs(source_current)
```

i. Create blink_source.c(For Source current):

```
$ nano CMakeLists.txt
```



```
#include "pico/stdlib.h"

#define GPIO_PIN 14
#define GPIO_MASK (1u << GPIO_PIN)

// SIO register base and offsets
#define SIO_BASE      0xD0000000
#define SIO_GPIO_OE    (*(volatile uint32_t *) (SIO_BASE + 0x20))
#define SIO_GPIO_OUT_CLR (*(volatile uint32_t *) (SIO_BASE + 0x18))
#define SIO_GPIO_OUT_SET (*(volatile uint32_t *) (SIO_BASE + 0x14))

int main() {
    stdio_init_all();

    // Enable GPIO14 as output
    SIO_GPIO_OE |= GPIO_MASK;

    while (1) {
        // Set GPIO14 HIGH → LED ON
        SIO_GPIO_OUT_SET = GPIO_MASK;
        sleep_ms(500);

        // Set GPIO14 LOW → LED OFF
        SIO_GPIO_OUT_CLR = GPIO_MASK;
        sleep_ms(500);
    }
}
```

4. APPENDIX:

1. <https://datasheets.raspberrypi.com/rp2040/rp2040-datasheet.pdf>
2. [pico-sdk/src/rp2_common/pico_stdio/include/pico/stdio.h at master · raspberrypi/pico-sdk · GitHub](#)